La aventura del sabio

TRABAJO GRUPAL

TEMA: JUEGO DE ROL

INTEGRANTES:

BENITES CORAZON, MAX ANDERSON 50%

CARLOS CAJO, JOSE MANUEL 50%

DOCENTE: ESPINOZA PAREDES, CHRISTIAN EDUARDO

SECCION: 38257

Versión 2.0
Mayo 2025

Historial de Versiones

VERSIÓN	PARTES QUE CAMBIAN	DESCRIPCIÓN DEL CAMBIO	FECHA DE CAMBIO	MODIFICADO POR	APROBADO POR
1.0			04/04/2025	Max BenitesC	
2.0			**25/04/2025**	Max Benites C	
3.0			16/06/2025	Max Benites C	

Índice

HISTORIAL DE VERSIONES 2
ÍNDICE 3
1 INTRODUCCIÓN 4
2 ALCANCE 5
3 CRONOGRAMA DE TRABAJO INICIAL 6
4 CRONOGRAMA DE TRABAJO FINAL 7
5 ACTORES DEL SISTEMA 7
6 REQUERIMIENTOS DE ALTO NIVEL 8
7 REQUERIMIENTOS FUNCIONALES 9

1. INTRODUCCIÓN

El presente proyecto grupal consiste en el diseño y desarrollo de un videojuego educativo de tipo rol titulado “La Aventura del Sabio”, programado en lenguaje Java. Este juego busca reforzar el aprendizaje de conocimientos generales a través de un sistema de preguntas y respuestas que se adapta en dificultad conforme el jugador progresa en niveles.

Cada jugador asume el rol de un “sabio” con una clase elemental (fuego, agua o tierra) y debe superar rondas de preguntas clasificadas en niveles: fácil, medio y difícil. A medida que aciertan preguntas, ganan puntos de experiencia (XP) que les permiten subir de nivel, hasta alcanzar el objetivo máximo: completar la misión.

El sistema se desarrolla como una aplicación de consola, diseñada para 1 a 3 jugadores locales en un entorno por turnos. Este trabajo permite aplicar los conocimientos aprendidos en programación estructurada, control de flujo, validación de entradas, uso de arreglos, y desarrollo lógico de videojuegos básicos.

Roles dentro del equipo:

- **Delegado**: **AGUILAR ROJAS, JEREMI ALDAIR**
  Será el responsable de coordinar al equipo, asignar tareas, organizar reuniones, y monitorear el cumplimiento del plan de trabajo. Será el punto de contacto principal entre el grupo y el docente.
- **Primer oficial**: **BENITES CORAZON, MAX ANDERSON**
  Actuará como segundo al mando. Su rol será garantizar que el plan de trabajo se cumpla, respaldar al delegado en sus funciones y documentar las reuniones del grupo, especialmente aquellas relacionadas con presentaciones o feedback del docente.
- **Imagen: MARTINEZ MARQUINA, FERNANDO FRANCISCO**
  Tendrá a su cargo la preparación de la presentación del proyecto. Se encargará de planificar los tiempos, definir los temas a tratar, y coordinar la forma en la que el grupo mostrará el trabajo realizado de manera clara y atractiva.
- **QA (Quality Assurance)**: **GASPAR BOTTONI, MIGUEL ANGEL**
  Su función será probar el código desarrollado por el equipo, asegurarse de que las funcionalidades se comporten correctamente y definir los casos de prueba que se usarán durante las presentaciones.
- **Integrador: CARLOS CAJO, JOSE MANUEL**
  Será responsable de unificar el código de los distintos integrantes, resolver conflictos de integración, y mantener el proyecto central funcionando correctamente.

2. ALCANCE

Este proyecto se enfoca en la creación de un videojuego educativo por turnos, en el que participan entre 1 y 3 jugadores, quienes responden preguntas clasificadas por dificultad, acumulando experiencia y progresando de nivel. El objetivo final es alcanzar el nivel máximo respondiendo correctamente la mayor cantidad de preguntas posibles.

2.1 Delimitaciones del sistema:

- El sistema será ejecutado como una aplicación de consola en Java.
- No se contempla el uso de interfaz gráfica (solo uso de texto por consola).
- El juego admite de 1 a 3 jugadores en una misma sesión.
- Las preguntas estarán almacenadas en arreglos (no se cargan desde archivos externos).
- Cada jugador tiene nombre, clase, puntos de experiencia (XP), nivel, aciertos y errores.
- La lógica de juego está organizada en turnos rotativos.
- Se controlan errores de entrada y flujo de juego de manera segura.
- No se implementará persistencia de datos (base de datos o archivos).

2.2 Tema 1: Módulo de gestión de jugadores

- Solicitar la cantidad de jugadores (1 a 3).
- Registrar el nombre de cada jugador.
- Permitir seleccionar una clase de personaje (fuego, agua o tierra).
- Asignar atributos iniciales según la clase elegida.
- Mostrar el nombre del jugador al inicio de cada turno.
- Validar los datos ingresados por el usuario y controlar errores.

2.3 Tema 2: Módulo de preguntas y validación

- Clasificar preguntas en tres niveles de dificultad: fácil, media y difícil.
- Asignar preguntas únicas a cada jugador sin repetir.
- Presentar opciones múltiples (A, B, C) para cada pregunta.
- Validar la respuesta ingresada y mostrar retroalimentación inmediata.
- Indicar la respuesta correcta si el jugador se equivoca.

2.4 Tema 3: Módulo de progreso, experiencia y resultados

- Otorgar puntos de experiencia (XP) por cada respuesta correcta.
- Calcular el nivel del jugador con base en la XP acumulada.
- Anunciar cuando el jugador sube de nivel y mostrar su nivel al finalizar la ronda.
- Contabilizar aciertos y errores por jugador.
- Mostrar resumen final por jugador con XP, nivel, aciertos y errores.
- Identificar a los jugadores que completaron la misión (nivel máximo).
- Anunciar ganadores o mostrar mensaje si ningún jugador logró completar la misión.

2.5 Tema 4: Módulo de interfaz y narrativa del juego

- La interfaz será por consola.
- Mostrar mensajes temáticos según la clase del jugador (fuego, agua, tierra).
- Presentar mensajes narrativos al subir de nivel o completar la misión.
- Indicar al jugador el turno actual, nivel y resultados finales.

3. CRONOGRAMA DE TRABAJO INICIAL

Semana	Fecha	Actividad	Estado	Responsable
Semana 2	4-Abr	Definir los roles dentro del grupo	Realizado	TODOS
Semana 2	4-Abr	Elaborar cronograma del trabajo inicial	Realizado	Delegado
Semana 2	4-Abr	Definir días de reunión	Realizado	Primer Oficial
Semana 2	5-Abr	Definir canales y horarios de comunicación	Realizado	Delegado
Semana 2	5-Abr	Primera reunión oficial	Realizado	Primer Oficial
Semana 3	5-Abr	Finalizar documento de introducción y alcance	Realizado	Delegado + QA
Semana 3	7-Abr	Avance del Proyecto al 30%	Realizado	TODOS
Semana 4	19-04 y 20-04	Diseño de personajes y mecánica del juego	Realizado	TODOS
Semana 5	26-04 y 27-04	Implementar módulo de creación de personajes	Realizado	Integrador
Semana 6	19-04 y 20-04	Implementar turnos y acciones básicas	Realizado	Integrador
Semana 7	26-04 y 27-04	Implementar sistema de enemigos y preguntas	Realizado	QA + Integrador
Semana 8	03-05 y 04-05	Pruebas iniciales del flujo del juego	Realizado	QA
Semana 8	10-05 y 11-05	Avance del Proyecto al 50%	Realizado	TODOS
Semana 9	17-05 y 18-05	Revisión de errores, mejoras	Realizado	TODOS
Semana 10	24-05 y 25-05	Añadir condiciones de victoria y validaciones	Realizado	Integrador
Semana 11	31-05 y 01-06	Implementar puntajes y estadísticas	Realizado	QA
Semana 12	07-06 y 08-06	Pruebas del sistema completo	Realizado	QA
Semana 13	14-06 y 15-06	Avance del Proyecto al 70%	Realizado	TODOS
Semana 13	21-06 y 22-06	Preparar presentación del sistema	Pendiente	Imagen
Semana 14	28-06 y 29-06	Ensayo de presentación	Pendiente	TODOS
Semana 15	05-07 y 06-07	Corrección de observaciones del docente	Pendiente	Primer Oficial

Semana 16	12-07 y 13-07	Revisión final y documentación completa	Pendiente	Delegado
Semana 17	19-07 y 20-07	Entrega y Exposición final del proyecto	Pendiente	TODOS

4. CRONOGRAMA DE TRABAJO FINAL

Fecha Estimada	Fecha Real	Actividad	Estado	Responsable
Semana 1	Semana 1	Formación del grupo	Terminado	Todos
Semana 3	Semana 3	Definición de roles	Terminado	Delegado
Semana 4	Semana 4	Diseño del juego	Terminado	Primer Oficial
Semana 6	Semana 6	Código base	Terminado	Primer Oficial
Semana 8	Semana 8	Redacción de documentación base	Terminado	Delegado
Semana 9	Semana 9	Implementación multijugador	En proceso	Todos
Semana 10	—	Retroalimentación docente	Pendiente	Primer Oficial
Semana 11	—	Prototipos para presentación	Pendiente	Imagen

5. ACTORES DEL SISTEMA

Actor	Descripción
Jugador(es)	El jugador es el participante principal del juego. Puede ingresar un nombre, elegir un personaje, resolver preguntas, subir de nivel y convertirse en sabio. Cada jugador tiene un límite de preguntas para ganar experiencia y llegar al nivel máximo. Gana aquel jugador que llegue alcanzar el nivel máximo.

6. REQUERIMIENTOS DE ALTO NIVEL

Detallar los requerimientos de alto nivel del sistema. **(6 por integrante)**

Requerimiento de Alto Nivel	Descripción
REQMS-001	Inicio del juego y selección de modo
REQMS-002	Selección y validación de cantidad de jugadores
REQMS-003	Diseño de la Clase Jugador
REQMS-004	Registro de Jugadores y Elección de Rol
REQMS-005	Diseño de la Clase Preguntas
REQMS-006	Inicialización del Banco de Preguntas
REQMS-007	Inicio del Sistema de Turnos
REQMS-008	Mostrar Estado del Jugador
REQMS-009	Mostrar Pregunta
REQMS-010	Mostrar Opciones de Respuesta
REQMS-011	Ingreso de Respuesta
REQMS-012	Validación de Respuesta
REQMS-013	Recompensa por Respuesta Correcta
REQMS-014	Bonificación por Racha de 3
REQMS-015	Ataque en Modo Versus
REQMS-016	Penalización por Respuesta Incorrecta
REQMS-017	Evaluación de Eliminación
REQMS-018	Final del Juego y Ganador

7. REQUERIMIENTOS FUNCIONALES

Código de Requerimiento	Descripción del Requerimiento Funcional	Propuesta de Solución
REQMS-001	Inicio del juego y selección de modo	1. Mostrar en consola el menú principal con tres opciones: Solitario, Versus y Salir. 2. Leer la opción ingresada por el usuario mediante Scanner. 3. Validar si la opción ingresada es válida (1, 2 o 9). 4. Si la opción es inválida, mostrar mensaje de error y volver a pedir opción. 5. Si elige 9, imprimir mensaje de despedida y finalizar el programa.
		Actores
		Jugador
		Alcance No Contemplado
		Nodo de dificultad(easy,medium,hard,leyend)
REQMS-002	Selección y validación de cantidad de jugadores	1. Si elige el modo Solitario, establecer numJugadores = 1. 2. Si elige el modo Versus, solicitar al usuario cuántos jugadores participarán (2 o 3). 3. Validar la entrada: solo se acepta 2 o 3. 4. Si la cantidad ingresada es inválida, repetir la solicitud hasta obtener una válida.
		Actores
		Jugador
		Alcance No Contemplado
		No se permite 4 o más jugadores

REQMS-003	Diseño de la Clase Jugador	1. Se crea la clase Jugador para representar a cada participante. 2. Se declaran atributos privados: nombre, rol, ps, energía, xp, racha y eliminado. 3. Se implementa un constructor que inicializa estos atributos con valores predeterminados según el rol elegido. 4. Se desarrollan métodos públicos como: a) sumarXp(int) b) recibirDano(int) c) curar(int) d) eliminar() e) getNombre() f) getRol() g) getPs() h) getXp() i) estaEliminado() j) getRacha() k) sumarRacha() l) reiniciarRacha() m) NombreAtaque() 5. La clase encapsula la lógica de comportamiento individual de los jugadores.
		Actores
		Sin actor, controlado por el SISTEMA
		Alcance No Contemplado
		Una opción de saltar pregunta
REQMS-004	Registro de Jugadores y Elección de Rol	1. Para cada jugador: a. Solicitar nombre. b. Mostrar opciones de rol (Fuego, Agua, Tierra). c. Leer y validar la elección del rol. d. Convertir la opción numérica en su correspondiente String (fuego, agua, tierra). e. Crear objeto `Jugador` con nombre y rol. f. Mostrar los atributos iniciales según el rol elegido.
		Actores
		Jugador
		Alcance No Contemplado
		No se validan nombres repetidos

REQMS-005	Diseño de la Clase Preguntas	1. Se crea la clase Preguntas para administrar preguntas, opciones y respuestas. 2. Se declaran tres arreglos privados: a) String[] preguntas: almacena las preguntas del juego. b) String[][] opciones: contiene las opciones tipo test para cada pregunta. c) String[] respuestas: define la letra correcta para cada pregunta. 3. Se implementa el constructor para inicializar esos arreglos. 4. Métodos definidos: a) mostrarOpciones(int index) b) getPregunta(int index) c) getRespuesta(int index) d) totalPreguntas() **Actores** Sin actor, controlado por el SISTEMA **Alcance No Contemplado** No se esta contemplando el extraer preguntas de fuente externa
REQMS-006	Inicialización del Banco de Preguntas	1. Se crearan objetos tipo Preguntas. 2. Al construir el objeto, automáticamente se cargan preguntas, opciones y respuestas. 3. Se garantiza que el sistema tenga preguntas accesibles para el modo Solitario y Versus. **Actores** Sin actor, controlado por el SISTEMA **Alcance No Contemplado** No se esta contemplando el extraer preguntas de fuente externa
REQMS-007	Inicio del Sistema de Turnos	1. Se establece una variable turnoGeneral para controlar los turnos globales. 2. Se determina un máximo de turnos (MAX_TURNOS). 3. Dependiendo del modo, se ejecuta un bucle diferente: a) Modo Solitario: bucle individual para un solo jugador. b) Modo Versus: bucle para múltiples jugadores. **Actores** Sin actor, controlado por el SISTEMA **Alcance No Contemplado** No se esta contemplando el extraer preguntas de fuente externa

REQMS-008	Mostrar Estado del Jugador	1. Antes de responder, se muestra: a) Nombre del jugador en turno. b) PS (Puntos de Salud) y XP (Experiencia) actuales. **Actores** Sin actor, controlado por el SISTEMA **Alcance No Contemplado**
REQMS-009	Mostrar Pregunta	1. Obtener la pregunta correspondiente al índice de turno. 2. Mostrar la pregunta en consola.0 **Actores** Sin actor, controlado por el SISTEMA **Alcance No Contemplado**
REQMS-010	Mostrar Opciones de Respuesta	1. Usar un metodo para mostrar opciones. 2. Mostrar en pantalla las 4 posibles respuestas (a, b, c y d). **Actores** Sin actor, controlado por el SISTEMA **Alcance No Contemplado**
REQMS-011	Ingreso de Respuesta	1. Solicitar al jugador que ingrese su respuesta (una letra). 2. Convertirla a minúscula y extraer el primer carácter (char). **Actores** Jugador **Alcance No Contemplado** No permite cambiar de respuesta una vez elegida una opcion
REQMS-012	Validación de Respuesta	1. Obtener la respuesta correcta usando get de la clase Pregunta para obtener respuesta. 2. Comparar la letra ingresada con la respuesta correcta. **Actores** Sin actor, controlado por el SISTEMA **Alcance No Contemplado**